Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Quantum Computing Fundamentals for Beginners

Course Code: C882

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	2 October 2026	Initial release of the Quantum Computing Fundamentals for Beginners learner guide: detailed step-by-step instructions for all 17 hands-on labs on IBM Quantum Composer, with the OpenQASM listing and a verification check for every lab, plus setup, troubleshooting and a glossary.	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Introduction **4**
Course Learning Outcomes **4**
Before You Start — Setting Up IBM Quantum Composer **4**
Topic 01 — Introduction to Quantum Computing **6**
Lab 1 — Explore the Qubit on the Bloch Sphere 9
Lab 2 — Observe Superposition and Interference 10
Lab 3 — Create Your First Entangled Pair (Bell State) 11
Topic 02 — Quantum States and Gates **12**
Lab 4 — Single-Qubit Gates — Pauli X, Y, Z and Hadamard 14
Lab 5 — Rotation and Phase Gates — RX, RY, RZ, S and T 15
Lab 6 — Two-Qubit Gates — CNOT, CZ, SWAP and Toffoli 16
Lab 7 — Bell States, GHZ States and Phase Kickback 17
Topic 03 — Quantum Algorithms **19**
Lab 8 — Quantum Teleportation 21
Lab 9 — Superdense Coding 22
Lab 10 — BB84 Quantum Key Distribution 24
Lab 11 — The Deutsch and Deutsch-Jozsa Algorithms 25
Lab 12 — The Bernstein-Vazirani Algorithm 27
Lab 13 — Grover's Search Algorithm 28
Topic 04 — Quantum Computing Applications **30**
Lab 14 — Quantum Error Correction — The Bit-Flip Code 31
Lab 15 — Quantum Error Correction — The Phase-Flip Code 33
Lab 16 — Simulator vs Real Quantum Hardware 34
Lab 17 — Introduction to Quantum Machine Learning 35
Troubleshooting — Common Problems **36**
Glossary **37**

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the course Quantum Computing Fundamentals for Beginners (C882), conducted by Tertiary Infotech Academy Pte Ltd (UEN 201200696W). It provides the full step-by-step instructions for all 17 hands-on labs, organised by the four course topics. Every lab is completed in your web browser on IBM Quantum Composer — no software installation and no cost.

This guide is your primary reference during the course and afterwards, whenever you rebuild a circuit on your own. Where the slides give you the concept and the picture, this guide gives you the exact sequence of gates to place, the OpenQASM that should result, and the outcome you must be able to verify.

No prior physics or quantum-mechanics background is assumed. Where mathematics appears it is explained in plain language first, and every idea is immediately followed by a circuit you build yourself.

Course Learning Outcomes

- LO1: Identify and use appropriate quantum computing platforms for testing quantum gates and algorithms
- LO2: Utilise and conduct quantum gates and logic to derive solutions
- LO3: Perform coding and conduct testing of quantum algorithms
- LO4: Diagnose outcomes, propose, and implement changes for quantum computing applications

Before You Start — Setting Up IBM Quantum Composer

What you need

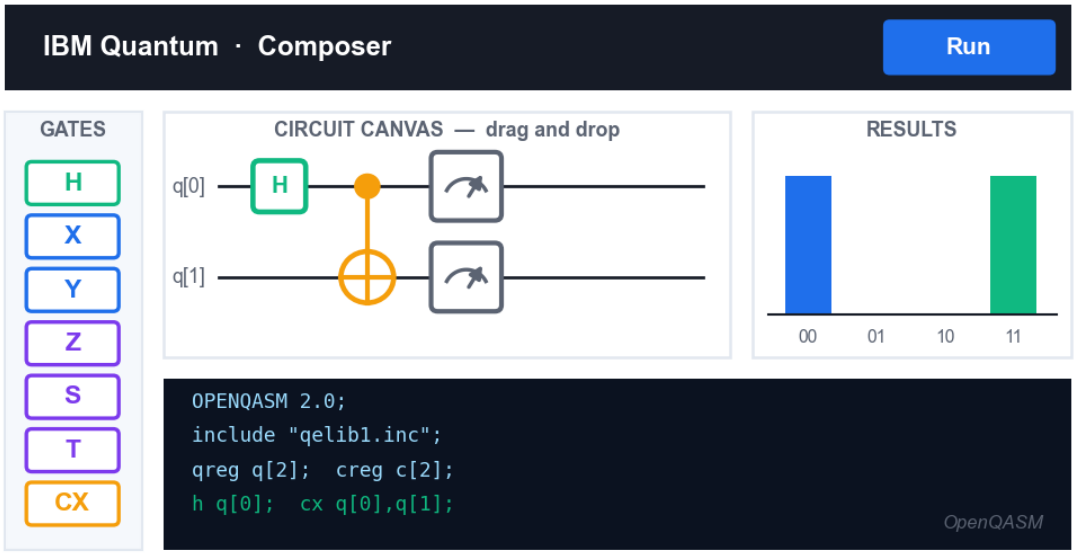
- A laptop or desktop with a modern web browser (Chrome, Edge, Firefox or Safari) and internet access.
- A free IBM Quantum account — sign up at https://quantum.ibm.com/ with any e-mail address.
- No software installation, no Python and no payment are required for any lab in this course.
- Optional supplementary tools: the Bloch sphere simulator (https://bloch.kherb.io/), Quirk (https://algassert.com/quirk) and the Quantum Circuit Designer (https://thequantumlaend.de/quantum-circuit-designer/).

Create your account and open the Composer

1. Go to https://quantum.ibm.com/ and select Create an IBMid account (or sign in if you already have one). Verify your e-mail address.
2. Sign in to the IBM Quantum Platform. You will land on the dashboard showing your available backends and recent jobs.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

3. Open the Composer at https://quantum.cloud.ibm.com/composer. This is the drag-and-drop circuit editor used for every lab in this course.
4. Familiarise yourself with the four panels: the GATE PALETTE on the left, the CIRCUIT CANVAS in the centre, the OPENQASM code editor below it, and the RESULTS/statevector panel on the right.
5. Create a new circuit and set the number of qubits using the circuit-width control. Most labs use 1 to 5 qubits.
6. Confirm you can run a job: place a single H gate and a Measure on q[0], click Setup and run, choose the SIMULATOR backend with 1024 shots, and read the resulting histogram.



The IBM Quantum Composer: gate palette, circuit canvas, OpenQASM editor and results histogram.

Conventions used in every lab

- Qubits are numbered from q[0]. Classical bits are numbered from c[0].
- All qubits start in the state |0> — you never have to initialise them.
- Gates are placed by dragging from the palette onto a wire; drag a gate off the wire to delete it.
- For a controlled gate, the SOLID DOT is the control and the circled cross (or dot, for CZ) is the target.
- A BARRIER is a visual separator that also prevents the compiler from optimising across it — use it to keep the sections of your circuit (encode / oracle / decode) clearly distinct.
- Every lab shows the OpenQASM that your circuit should produce. You can paste this directly into the OpenQASM editor instead of dragging gates, and the canvas will update to match.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Unless stated otherwise, run on the SIMULATOR with 1024 shots. Only Lab 16 requires real quantum hardware.

Reading a results histogram

- Each bar is one measurement outcome, labelled with the classical bit string; the height is the fraction of shots.
- Bit ordering: IBM writes the string with the HIGHEST-numbered classical bit on the LEFT. For a 2-qubit circuit the label reads c[1]c[0], so the outcome '10' means c[1]=1 and c[0]=0.
- At 1024 shots a true 50% outcome typically lands between about 47% and 53% — small variation is normal sampling noise, not an error.
- A result that should be deterministic (100%) will be exactly 100% on the simulator, but only approximately so on real hardware.

Note: If a circuit will not run, the most common causes are a missing Measure operation, a classical register that is too small for the number of measured qubits, or a control and target placed on the same wire.

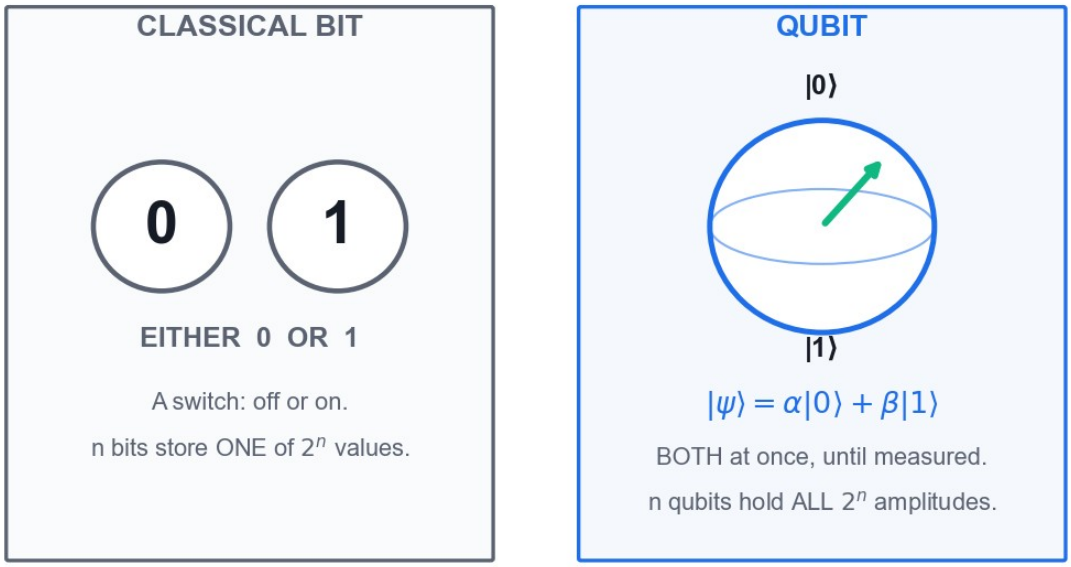
Topic 01 — Introduction to Quantum Computing

Key Principles of Quantum Computing · Quantum Computing Platforms

Key concepts

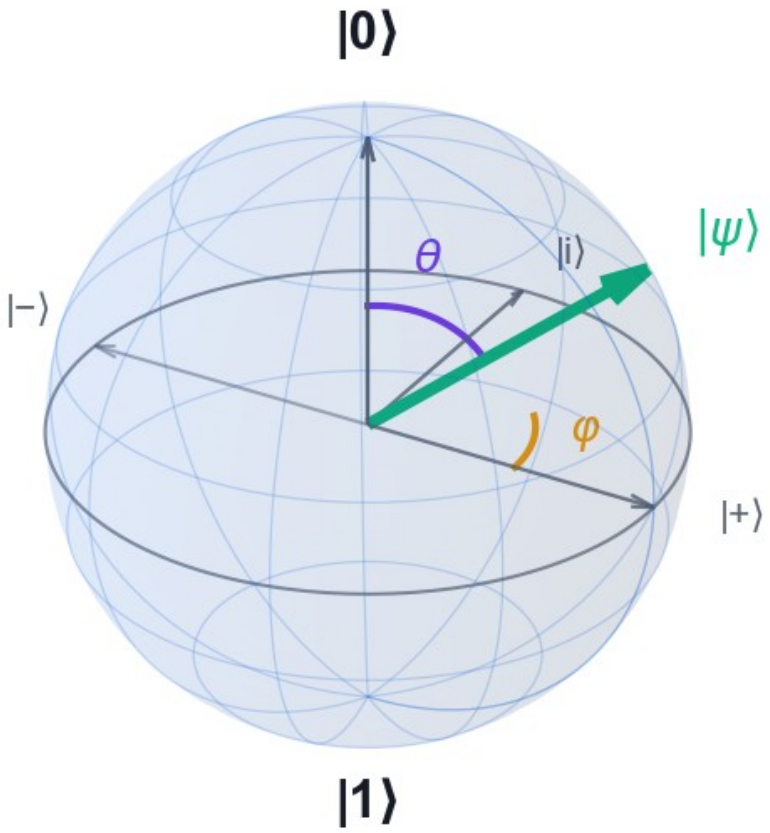
- **What is a quantum computer —** A machine that computes on qubits using two laws of the quantum world: superposition and entanglement.
- **The qubit —** A classical bit is 0 or 1. A qubit can be |0>, |1>, or a superposition of both at the same time.
- **The Bloch sphere —** Any single-qubit pure state is a point on a sphere — the geometric picture behind every gate you apply.
- **Physical qubits —** Superconducting (Josephson junction), trapped ion, photonic and topological — any two-level quantum system.
- **Superposition & interference —** Amplitudes add and cancel. Interference is what a quantum algorithm engineers to make wrong answers vanish.
- **Entanglement —** Linked particles whose states cannot be described independently — Einstein's 'spooky action at a distance'.
- **Postulates of quantum mechanics —** State space, unitary time evolution, observables, the Born rule and wave-function collapse.
- **Platforms —** IBM Quantum Composer and Platform, Quirk, the Quantum Circuit Designer, PennyLane and Qiskit.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



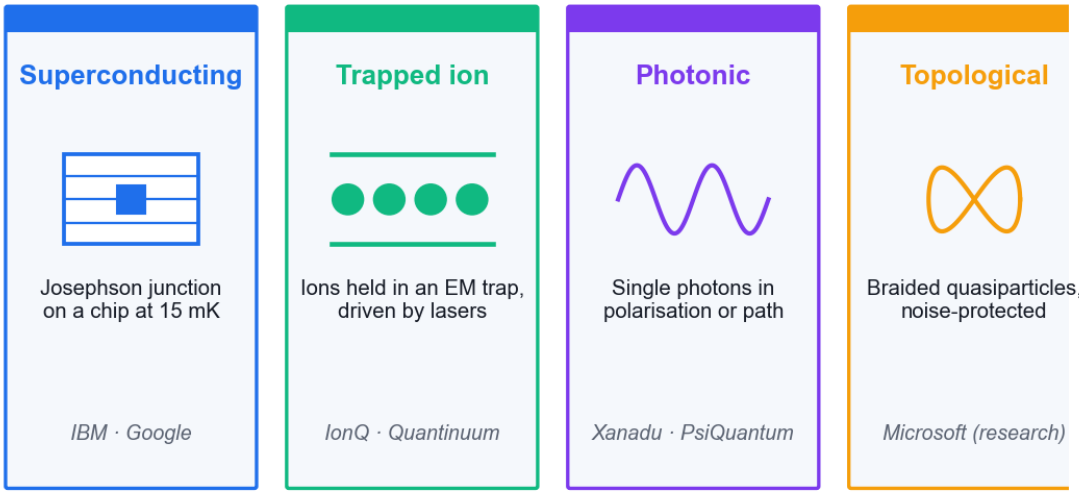
A classical bit is 0 or 1; a qubit is a superposition of both.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



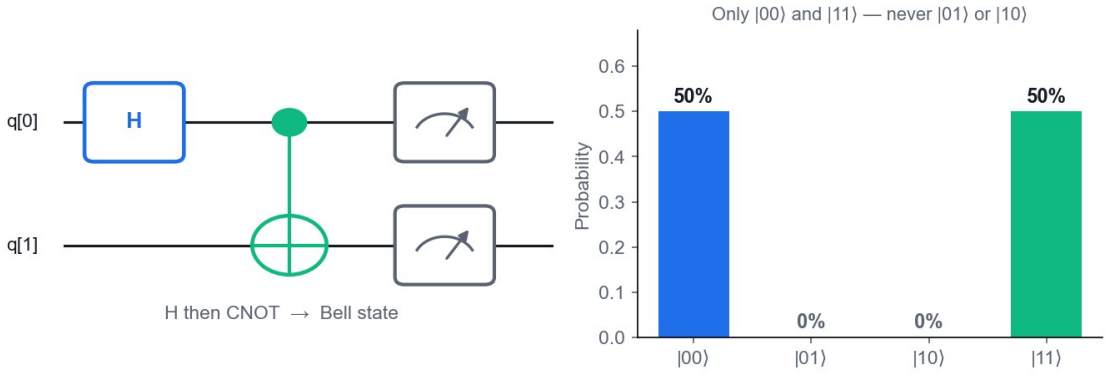
The Bloch sphere: any single-qubit pure state is a point on the surface.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Four physical realisations of a qubit.

Entanglement: (|00⟩ + |11⟩)/√2



The Bell state — only |00> and |11> ever appear.

Lab 1 — Explore the Qubit on the Bloch Sphere

Learning outcome: LO1 — identify and use appropriate quantum computing platforms.

Goal: Use an interactive Bloch-sphere simulator to see what a qubit actually is: a point on a sphere. Apply X, H and Z and watch the state vector move, so that every gate you meet later has a picture attached to it.

What you'll build

A worked set of Bloch-sphere states (|0>, |1>, |+>, |->) and the gates that produce them. (Platform: Bloch Sphere Simulator (bloch.kherb.io) — https://bloch.kherb.io/)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

7. Open the Bloch sphere simulator at https://bloch.kherb.io/ and identify the three axes. The north pole is |0> and the south pole is |1>.
8. Confirm the qubit starts at the north pole — the state |0>, which measures as 0 with probability 1.
9. Apply the X gate. The vector rotates 180 degrees about the X axis to the south pole: X|0> = |1>.
10. Reset, then apply H. The vector moves to the +X equator — the state |+> = (|0> + |1>)/sqrt(2), a 50/50 superposition.
11. With the state at |+>, apply Z. The vector rotates to -X, giving |-> = (|0> - |1>)/sqrt(2). Note the probabilities did NOT change — only the relative phase.
12. Apply H a second time from |+>. The vector returns to |0>, showing that H is its own inverse (H.H = I).
13. Record for each state the polar angle theta and azimuth phi, and relate them to |psi> = cos(theta/2)|0> + e^(i.phi).sin(theta/2)|1>.

Test it — how to verify you succeeded

You can place |0>, |1>, |+> and |-> on the sphere from memory, and state which gate maps each one to the next. You can explain why Z changes |+> to |-> without changing any measurement probability.

Note: The OpenQASM for this circuit is also saved in labs/lab-01-explore-the-qubit-on-the-bloch-sphere/.

Lab 2 — Observe Superposition and Interference

Learning outcome: LO1 — identify key principles of quantum computing.

Goal: Build the smallest possible quantum circuit — a single Hadamard gate — on IBM Quantum Composer, run it on the simulator, and read the probability histogram. Then add a second Hadamard to see interference cancel the superposition completely.

What you'll build

Two single-qubit circuits: H (a 50/50 random bit) and H-H (a deterministic 0), with their histograms. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

14. Sign in to IBM Quantum at https://quantum.cloud.ibm.com/ and open the Composer from the left-hand menu.
15. Create a new circuit and set the circuit width to 1 qubit and 1 classical bit.
16. Drag an H gate onto qubit q[0], then drag a Measure operation after it.
17. Open the OpenQASM editor and confirm the code reads: h q[0]; measure q[0] -> c[0];

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
h q[0];
measure q[0] -> c[0];
```

18. Run the circuit on the simulator with 1024 shots and read the probability histogram — you should see roughly 50% |0> and 50% |1>.
19. Now insert a SECOND H gate between the first H and the Measure, so the circuit is H-H-Measure.

```
h q[0];
h q[0];
measure q[0] -> c[0];
```

20. Run again with 1024 shots. The result is now 100% |0> — the two paths interfered destructively for |1> and constructively for |0>.

Test it — how to verify you succeeded

Circuit 1 gives approximately 50/50 across |0> and |1>; circuit 2 gives 100% |0>. You can explain that the second H did not 'undo randomness' — it caused interference, which is the resource quantum algorithms exploit.

Note: The OpenQASM for this circuit is also saved in labs/lab-02-observe-superposition-and-interference/.

Lab 3 — Create Your First Entangled Pair (Bell State)

Learning outcome: LO1/LO2 — use a quantum platform to demonstrate entanglement.

Goal: Build the Bell state (|00> + |11>)/sqrt(2) on IBM Quantum Composer with just H and CNOT, and confirm from the histogram that the two qubits are perfectly correlated — the signature of entanglement.

What you'll build

A 2-qubit Bell-state circuit whose histogram shows only |00> and |11>, each near 50%. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

21. Open a new circuit in IBM Quantum Composer with 2 qubits and 2 classical bits.
22. Place an H gate on q[0]. This puts the control qubit into superposition.

```
h q[0];
```

23. Place a CNOT with the control on q[0] and the target on q[1] — drag the CNOT so its solid dot sits on q[0].

```
cx q[0], q[1];
```

24. Add Measure operations on both q[0] and q[1].

```
measure q[0] -> c[0];
measure q[1] -> c[1];
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

25. Inspect the statevector panel: the amplitudes are 1/sqrt(2) on |00> and 1/sqrt(2) on |11>, and exactly zero on |01> and |10>.
26. Run on the simulator with 1024 shots. The histogram shows only two bars — |00> near 50% and |11> near 50%.
27. Reason about the correlation: no shot ever produces |01> or |10>, so measuring one qubit instantly determines the other.

Test it — how to verify you succeeded

Your histogram contains only |00> and |11>, each approximately 50%, with |01> and |10> essentially absent. You can explain why this state cannot be written as a product of two independent single-qubit states.

Note: The OpenQASM for this circuit is also saved in labs/lab-03-create-your-first-entangled-pair-bell-st/.

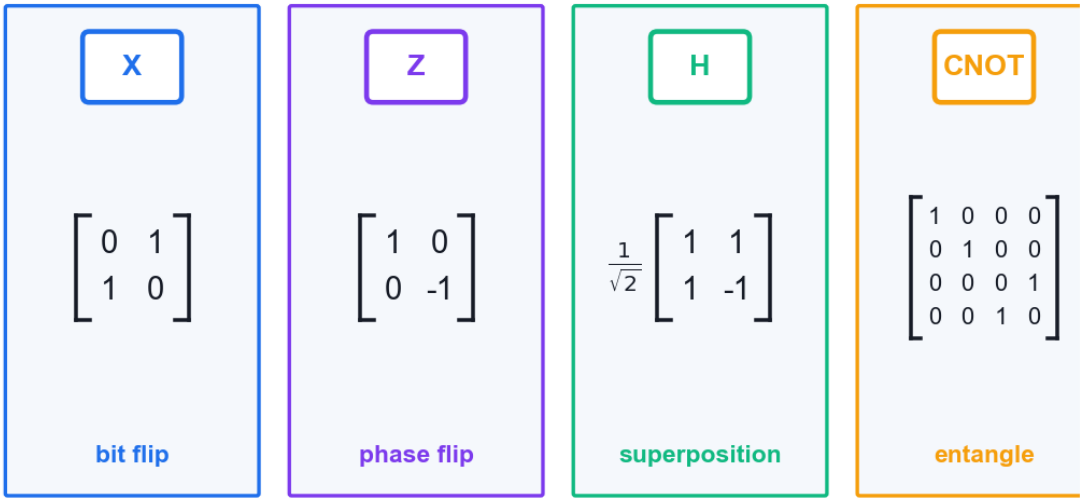
Topic 02 — Quantum States and Gates

Quantum State Vector Representations · Quantum Circuits and Quantum Gates

Key concepts

- **State vectors & Dirac notation —** |psi> = a|0> + b|1> with |a|^2 + |b|^2 = 1. Gates are unitary matrices acting on this vector.
- **Pauli gates X, Y, Z —** The quantum NOT, the bit-and-phase flip, and the phase flip. Each is its own inverse.
- **Hadamard gate —** Creates superposition: H|0> = |+>, H|1> = |->. The gate that starts almost every algorithm.
- **Rotation & phase gates —** RX, RY, RZ rotate the Bloch vector; P, S and T apply a relative phase to |1>.
- **CNOT, CZ and controlled gates —** Two-qubit gates that entangle. CNOT flips the target when the control is |1>.
- **SWAP and Toffoli —** SWAP exchanges two qubits; Toffoli (CCNOT) is universal for reversible classical computation.
- **Bell and GHZ states —** H then CNOT builds a Bell pair — the maximally entangled two-qubit state; GHZ extends it to n qubits.
- **Phase kickback —** A phase applied to the target of a controlled gate 'kicks back' onto the control — the engine of Deutsch, BV and QPE.

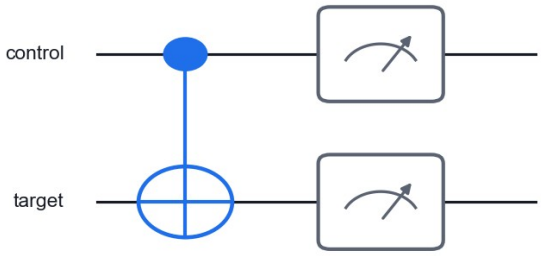
© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Every gate is unitary: $UU^\dagger = I$ — so every quantum operation is reversible.

Every quantum gate is a unitary matrix.

The CNOT Gate



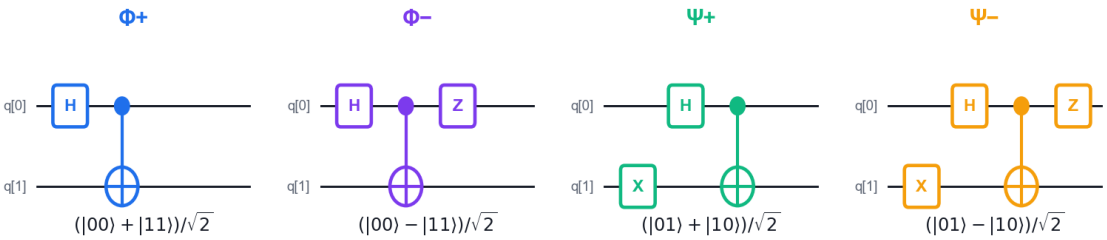
Flip the target iff the control is |1⟩

INPUT	OUTPUT
|00⟩	|00⟩
|01⟩	|01⟩
|10⟩	|11⟩
|11⟩	|10⟩

Flipped rows in teal

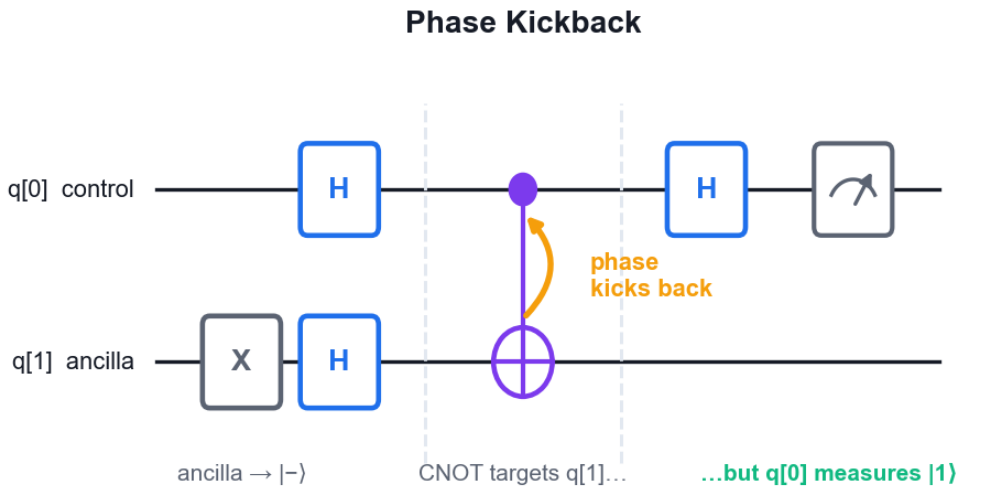
The CNOT gate flips the target only when the control is |1>.

The Four Bell States — an orthonormal, maximally entangled basis



© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The four Bell states and the circuits that build them.



$$\text{CNOT}\,|x\rangle|-\rangle = (-1)^x|x\rangle|-\rangle$$ — the answer becomes a PHASE on the control

Phase kickback: the target's phase appears on the control.

Lab 4 — Single-Qubit Gates — Pauli X, Y, Z and Hadamard

Learning outcome: LO2 — utilise quantum gates and logic to derive solutions.

Goal: Work through the single-qubit gate set on IBM Quantum Composer. Apply each Pauli gate and the Hadamard to |0> and to |1>, read the resulting statevector, and verify from the simulator that each Pauli gate is its own inverse.

What you'll build

A gate reference table you fill in yourself: the output state and measurement probabilities for X, Y, Z and H on |0> and |1>. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

28. Open a 1-qubit circuit in IBM Quantum Composer and keep the statevector panel visible.
29. Apply X to q[0] and confirm the state becomes |1> — X is the quantum NOT gate.

```
x q[0];
```

30. Clear, apply Z to q[0], and confirm the state stays |0>. Z acts only on the |1> component, so it has no visible effect here.

```
z q[0];
```

31. Build H then Z. The state moves |0> -> |+> -> |->. Z now flips the sign of the |1> amplitude — a relative phase, invisible to measurement.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
h q[0];
z q[0];
```

32. Apply Y to |0> and read the statevector: you get i|1>. Y combines a bit flip and a phase flip (Y = iXZ).

```
y q[0];
```

33. Verify self-inversion: build X-X, then Z-Z, then H-H. Each returns the qubit to |0>, confirming X^2 = Y^2 = Z^2 = H^2 = I.

```
x q[0];
x q[0];
```

34. Add a Measure to the H-Z circuit and run 1024 shots. The histogram is still 50/50 — proving a relative phase alone cannot be measured directly.

Test it — how to verify you succeeded

You can complete a table giving the output state for X, Y, Z and H applied to |0> and to |1>, and demonstrate on the Composer that applying any one of them twice returns the qubit to its starting state.

Note: The OpenQASM for this circuit is also saved in labs/lab-04-single-qubit-gates-—-pauli-x-y-z-and-had/.

Lab 5 — Rotation and Phase Gates — RX, RY, RZ, S and T

Learning outcome: LO2 — utilise quantum gates and logic to derive solutions.

Goal: Move beyond fixed gates to parameterised rotations. Sweep the angle of an RY gate on IBM Quantum Composer and watch the measurement probability change continuously — the basis of every variational quantum algorithm.

What you'll build

A probability-vs-angle table for RY(theta) at theta = 0, pi/4, pi/2, 3pi/4 and pi, plus the S and T phase gates demonstrated. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

35. Open a 1-qubit circuit with a Measure and place an RY gate on q[0].
36. Set theta = pi/2 and run 1024 shots. RY(pi/2) is equivalent to H up to phase, so you get approximately 50/50.

```
ry(pi/2) q[0];
```

37. Set theta = pi and run again. RY(pi) maps |0> to |1>, so the histogram is 100% |1>.

```
ry(pi) q[0];
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

38. Set theta = pi/4 and predict the split BEFORE running: P(0) = cos^2(theta/2) = cos^2(pi/8) which is about 85%. Run and confirm.

```
ry(pi/4) q[0];
```

39. Replace RY with RX(pi) and confirm it also produces |1> — the rotation axis differs but the endpoint here is the same up to phase.

```
rx(pi) q[0];
```

40. Build H then S then H, and run. The S gate is a 90-degree phase gate (P(pi/2)); sandwiched between Hadamards its phase becomes visible in the histogram.

```
h q[0];
s q[0];
h q[0];
```

41. Repeat with T (a 45-degree phase gate, P(pi/4)) in place of S, and compare the two histograms.

```
h q[0];
t q[0];
h q[0];
```

Test it — how to verify you succeeded

Your measured P(0) matches cos^2(theta/2) at each angle you tried, to within statistical noise at 1024 shots. You can explain why a phase gate only changes the histogram when it is placed between Hadamards.

Note: The OpenQASM for this circuit is also saved in labs/lab-05-rotation-and-phase-gates-—-rx-ry-rz-s-an/.

Lab 6 — Two-Qubit Gates — CNOT, CZ, SWAP and Toffoli

Learning outcome: LO2 — utilise quantum gates and logic to derive solutions.

Goal: Build the truth table of the multi-qubit gates by hand on IBM Quantum Composer. Prepare each computational basis input with X gates, apply the gate, and read off the output — the fastest way to make CNOT, CZ, SWAP and Toffoli concrete.

What you'll build

Completed truth tables for CNOT, SWAP and Toffoli, plus a demonstration that CZ only applies a phase. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

42. Open a 2-qubit circuit. Prepare the input |10> by placing an X gate on q[0], then apply CNOT with control q[0] and target q[1].

```
x q[0];
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cx q[0], q[1];
```

43. Measure both qubits and run. The output is |11> — the control was |1>, so the target flipped.
44. Repeat for input |00> (no X gates). The output stays |00>, because the control was |0>. Record all four rows of the CNOT truth table.

```
cx q[0], q[1];
```

45. Replace the CNOT with a CZ gate on input |11> (X on both qubits). The measured bits are unchanged — CZ applies a -1 phase, not a bit flip.

```
x q[0];
x q[1];
cz q[0], q[1];
```

46. Make the CZ phase visible: build H on q[1], then CZ, then H on q[1]. This converts CZ into a CNOT, and the bit flip now appears.

```
x q[0];
h q[1];
cz q[0], q[1];
h q[1];
```

47. Place a SWAP gate on input |10> and run. The output is |01> — the two qubit states have been exchanged.

```
x q[0];
swap q[0], q[1];
```

48. Open a 3-qubit circuit, prepare |110> with X on q[0] and q[1], and apply a Toffoli (CCX) with controls q[0], q[1] and target q[2]. The target flips only because BOTH controls are |1>.

```
x q[0];
x q[1];
ccx q[0], q[1], q[2];
```

Test it — how to verify you succeeded

Your CNOT and Toffoli truth tables are complete and correct for every basis input, and you can demonstrate that H-CZ-H on the target is equivalent to a CNOT.

Note: The OpenQASM for this circuit is also saved in labs/lab-06-two-qubit-gates-—-cnot-cz-swap-and-toffo/.

Lab 7 — Bell States, GHZ States and Phase Kickback

Learning outcome: LO2 — utilise quantum gates and logic to derive solutions.

Goal: Build all four Bell states and the 3-qubit GHZ state on IBM Quantum Composer, then demonstrate phase kickback — the mechanism that makes the Deutsch, Bernstein-Vazirani and phase-estimation algorithms work.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

What you'll build

The four Bell states, a GHZ state, and a phase-kickback circuit showing the phase moving onto the control qubit. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

49. Build the Phi+ Bell state with H on q[0] then CNOT(q[0], q[1]). Confirm the histogram shows only |00> and |11>.

```
h q[0];
cx q[0], q[1];
```

50. Add an X on q[1] before the CNOT to build Psi+ = (|01> + |10>)/sqrt(2). The histogram now shows only |01> and |10>.

```
h q[0];
x q[1];
cx q[0], q[1];
```

51. Add a Z on q[0] after the CNOT to build Phi- = (|00> - |11>)/sqrt(2). The histogram is unchanged, but the statevector panel shows the negative amplitude.

```
h q[0];
cx q[0], q[1];
z q[0];
```

52. Extend to three qubits: H on q[0], CNOT(q[0], q[1]), CNOT(q[0], q[2]). This is the GHZ state (|000> + |111>)/sqrt(2).

```
h q[0];
cx q[0], q[1];
cx q[0], q[2];
```

53. Run the GHZ circuit for 1024 shots and confirm only |000> and |111> appear — all three qubits are perfectly correlated.
54. Now demonstrate phase kickback. Build a 2-qubit circuit: X on q[1], then H on both qubits, then CNOT(q[0], q[1]), then H on q[0].

```
x q[1];
h q[0];
h q[1];
cx q[0], q[1];
h q[0];
```

55. Measure q[0] and run. You measure |1> with certainty, even though the CNOT targeted q[1] — the phase from the |-> target has kicked back onto the control.

Test it — how to verify you succeeded

You can produce at least three of the four Bell states and the GHZ state, and explain why the phase-kickback circuit deterministically measures 1 on the control qubit.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

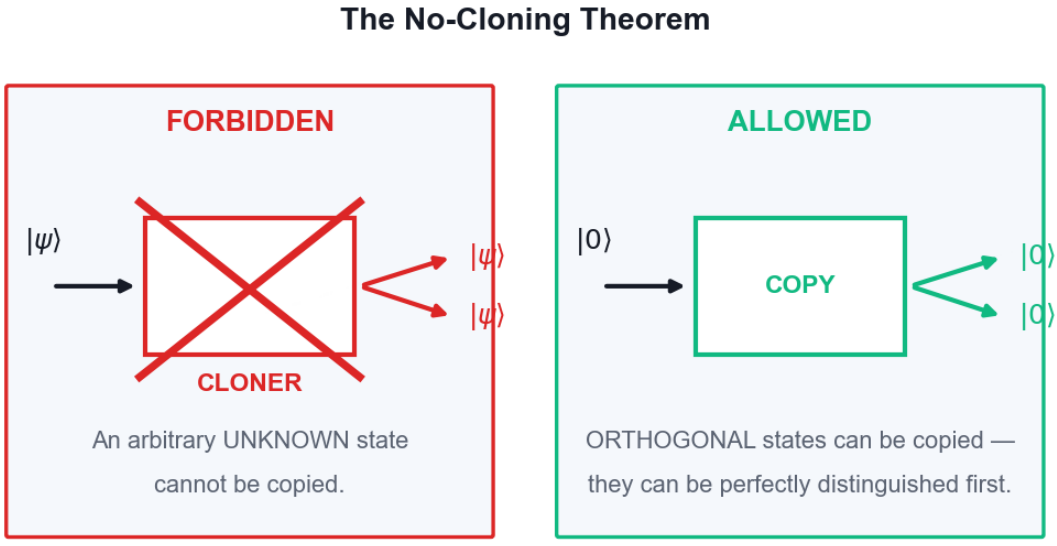
Note: The OpenQASM for this circuit is also saved in labs/lab-07-bell-states-ghz-states-and-phase-kickbac/.

Topic 03 — Quantum Algorithms

Coding Quantum Algorithms · Testing Quantum Algorithms

Key concepts

- **No-cloning theorem —** An arbitrary unknown quantum state cannot be copied — the reason quantum cryptography works.
- **Quantum teleportation —** Move an unknown qubit state using one shared Bell pair plus two classical bits.
- **Superdense coding —** The mirror image: send two classical bits by transmitting a single qubit of a shared Bell pair.
- **BB84 quantum key distribution —** Bennett & Brassard, 1984 — the first QKD protocol; eavesdropping shows up as errors in the sifted key.
- **The query (oracle) model —** Count how many times a black-box function must be asked. It isolates true quantum advantage.
- **Deutsch & Deutsch-Jozsa —** Decide constant vs balanced in ONE query where a classical computer may need 2^(n-1)+1.
- **Bernstein-Vazirani —** Recover an n-bit hidden string s from f(x) = s.x mod 2 in a single query instead of n.
- **Grover's search —** Quadratic speedup for unstructured search: O(sqrt(N)) instead of O(N), via oracle + diffusion.



© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

No-cloning: an arbitrary unknown state cannot be copied.

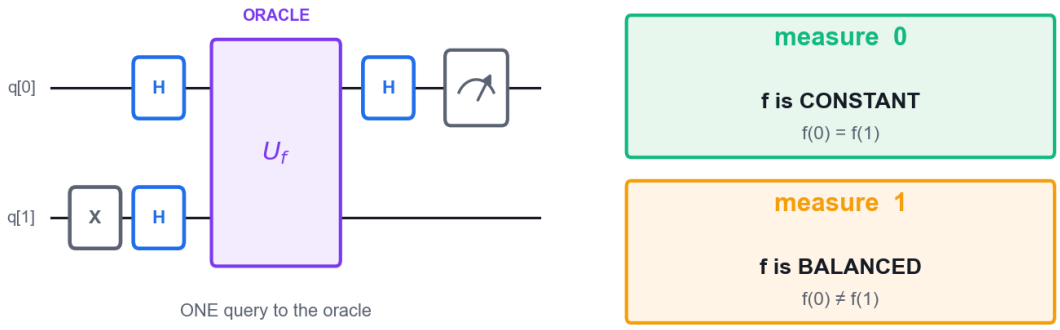
BB84 — sift the key by keeping only the matched-basis positions

	1	2	3	4	5	6	7	8
Alice's bit	0	1	1	0	1	0	0	1
Alice's basis	Z	X	Z	X	X	Z	X	Z
Bob's basis	Z	X	X	X	Z	Z	X	X
Bases match?	✓	✓	×	✓	×	✓	✓	×
Sifted key	0	1	—	0	—	0	0	—

Shared secret key = 01000 · an eavesdropper injects ~25% errors into these bits

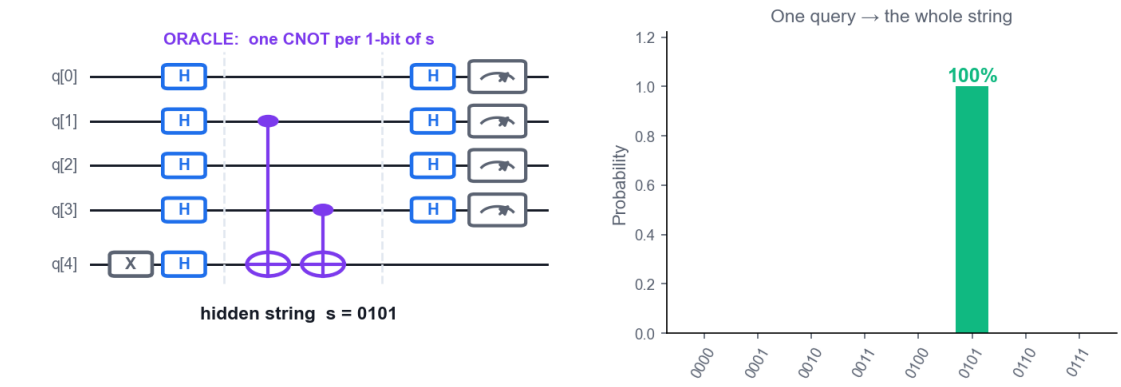
BB84 key sifting — keep only the matched-basis positions.

The Deutsch Algorithm — one query instead of two

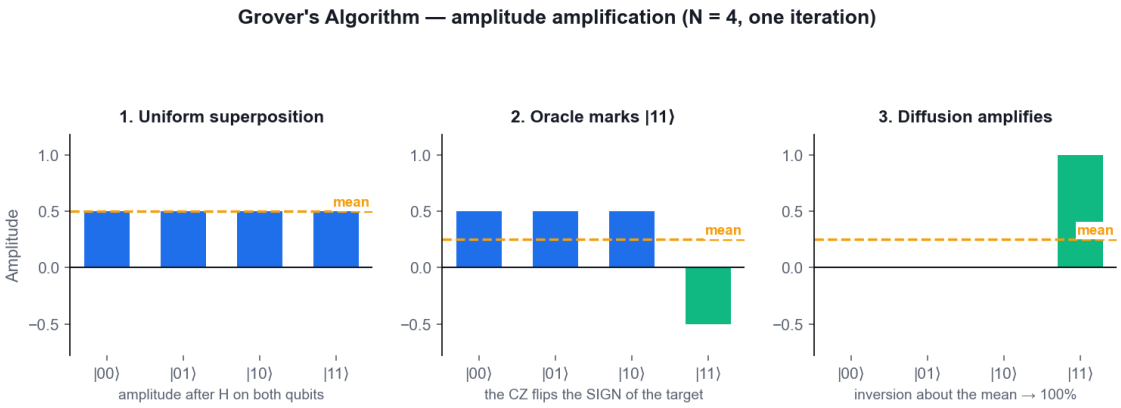


The Deutsch algorithm decides constant vs balanced in one query.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Bernstein-Vazirani recovers the whole hidden string in one query.



Grover's algorithm: amplitude amplification over three stages.

Lab 8 — Quantum Teleportation

Learning outcome: LO3 — perform coding and conduct testing of quantum algorithms.

Goal: Teleport an unknown qubit state from Alice to Bob on IBM Quantum Composer using one shared Bell pair and two classical bits. Build it until you can do it from memory.

What you'll build

A 3-qubit teleportation circuit that moves a |+> state from q[0] to q[2] and verifies the transfer. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

56. Open a 3-qubit circuit in IBM Quantum Composer. Qubit q[0] is Alice's message, q[1] is Alice's half of the Bell pair, and q[2] is Bob's half.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

57. Prepare the state to be teleported. Place an H on q[0] to create |+> = (|0> + |1>)/sqrt(2).

```
h q[0];
```

58. Create the shared Bell pair between Alice and Bob: H on q[1], then CNOT with control q[1] and target q[2].

```
h q[1];
cx q[1], q[2];
```

59. Add a barrier, then perform Alice's Bell measurement: CNOT with control q[0] and target q[1], followed by H on q[0].

```
cx q[0], q[1];
h q[0];
```

60. Measure q[0] and q[1] into the classical bits. These two bits are the classical message Alice sends to Bob.

```
measure q[0] -> c[0];
measure q[1] -> c[1];
```

61. Apply Bob's conditional corrections: a CNOT from q[1] to q[2] and a CZ from q[0] to q[2]. On the Composer these deferred-measurement gates reproduce the classically controlled X and Z.

```
cx q[1], q[2];
cz q[0], q[2];
```

62. Verify the teleportation: append an H to q[2] and measure it. Since the teleported state was |+>, H maps it back to |0>, so q[2] must read 0 on every shot.

```
h q[2];
measure q[2] -> c[2];
```

63. Run 1024 shots on the simulator and confirm the q[2] bit is 0 in every outcome, while q[0] and q[1] vary randomly across all four combinations.

Test it — how to verify you succeeded

Bob's qubit q[2] measures 0 on essentially every shot after the verification H, proving the |+> state arrived intact. You can explain why the no-cloning theorem is not violated: Alice's measurement destroys the original.

Note: The OpenQASM for this circuit is also saved in labs/lab-08-quantum-teleportation/.

Lab 9 — Superdense Coding

Learning outcome: LO3 — perform coding and conduct testing of quantum algorithms.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: Run teleportation in reverse: send two classical bits by transmitting one qubit. Encode each of the four messages 00, 01, 10 and 11 on IBM Quantum Composer and confirm Bob decodes each one with certainty.

What you'll build

Four superdense-coding circuits, one per two-bit message, each decoding deterministically. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

64. Open a 2-qubit circuit. Create the shared Bell pair first: H on q[0], then CNOT(q[0], q[1]).

```
h q[0];
cx q[0], q[1];
```

65. Add a barrier. Alice now encodes her two-bit message by acting on her qubit q[0] ONLY.
66. To send 00, Alice applies nothing (the identity). Add the decoder and run — see the next steps for the decoder.
67. Build the decoder that Bob applies: CNOT(q[0], q[1]) followed by H on q[0], then measure both qubits.

```
cx q[0], q[1];
h q[0];
measure q[0] -> c[0];
measure q[1] -> c[1];
```

68. Run the 00 circuit for 1024 shots. The histogram is 100% on a single outcome — Bob has recovered the message with certainty.
69. To send 01, insert an X gate on q[0] after the barrier. Re-run and note the new single deterministic outcome.

```
h q[0];
cx q[0], q[1];
x q[0];
cx q[0], q[1];
h q[0];
```

70. To send 10, use a Z gate on q[0] instead of X. To send 11, use both Z and X (i.e. the Y encoding). Run each and record the outcome.

```
h q[0];
cx q[0], q[1];
z q[0];
cx q[0], q[1];
h q[0];
```

71. Tabulate the four encodings (I, X, Z, ZX) against the four measured outcomes and confirm the mapping is one-to-one.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it — how to verify you succeeded

Each of the four encodings produces a different, single, deterministic measurement outcome at 1024 shots. You can explain that Alice transmitted only ONE qubit yet conveyed TWO classical bits, because entanglement was shared in advance.

Note: The OpenQASM for this circuit is also saved in labs/lab-09-superdense-coding/.

Lab 10 — BB84 Quantum Key Distribution

Learning outcome: LO3 — perform coding and conduct testing of quantum algorithms.

Goal: Simulate the BB84 key-exchange protocol on IBM Quantum Composer. Encode bits in randomly chosen bases, measure in randomly chosen bases, sift the key — then introduce an eavesdropper and watch the error rate expose her.

What you'll build

A sifted BB84 key produced without eavesdropping, and the elevated error rate produced when Eve intercepts. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

72. Understand the encoding: Alice sends bit 0 as |0> or |+>, and bit 1 as |1> or |->, choosing the Z or X basis at random for each qubit.
73. On a 1-qubit circuit, prepare bit 1 in the X basis: X then H, giving the state |->.

```
x q[0];
h q[0];
```

74. Bob measures in the X basis by applying H before the Measure. Run 1024 shots — he recovers bit 1 with certainty because the bases matched.

```
x q[0];
h q[0];
h q[0];
measure q[0] -> c[0];
```

75. Now let Bob measure in the WRONG basis: remove his H so he measures in Z. Run again — the result is 50/50 random, so this bit is useless and gets discarded during sifting.

```
x q[0];
h q[0];
measure q[0] -> c[0];
```

76. Repeat for at least 8 qubits with a random basis choice for Alice and for Bob. Record Alice's bit, Alice's basis, Bob's basis and Bob's result in a table.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

77. Sift the key: keep only the positions where Alice's and Bob's bases matched, and discard the rest. The retained bits are the shared secret key, and they should agree perfectly.
78. Now insert Eve. Between Alice and Bob, add an H followed by a Measure (Eve measuring in the X basis), then let the qubit continue to Bob.

```
x q[0];
h q[0];
measure q[0] -> c[0];
h q[0];
measure q[0] -> c[1];
```

79. Re-run the matched-basis cases with Eve present. Roughly 25% of the sifted bits now disagree — the error rate that reveals the eavesdropper.

Test it — how to verify you succeeded

Without Eve, every matched-basis bit agrees between Alice and Bob. With Eve intercepting, about a quarter of the sifted bits disagree. You can explain why the no-cloning theorem makes this detection unavoidable for an eavesdropper.

Note: The OpenQASM for this circuit is also saved in labs/lab-10-bb84-quantum-key-distribution/.

Lab 11 — The Deutsch and Deutsch-Jozsa Algorithms

Learning outcome: LO3 — perform coding and conduct testing of quantum algorithms.

Goal: Build the first algorithm that beats every classical method. Determine whether a black-box function is constant or balanced using a single query on IBM Quantum Composer, then scale it up to the multi-qubit Deutsch-Jozsa version.

What you'll build

Four Deutsch circuits (both constant and both balanced oracles) and a 3-qubit Deutsch-Jozsa circuit. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

80. Open a 2-qubit circuit. Set up the input register: X on q[1] to make it |1>, then H on both qubits.

```
x q[1];
h q[0];
h q[1];
```

81. Add a barrier, then build the BALANCED oracle f(x) = x as a single CNOT with control q[0] and target q[1].

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cx q[0], q[1];
```

82. Add a barrier, apply H to q[0], and measure q[0] only.

```
h q[0];
measure q[0] -> c[0];
```

83. Run 1024 shots. The result is 100% |1>, which is the algorithm's verdict: the function is BALANCED.
84. Replace the oracle with the CONSTANT function f(x) = 0 — an empty oracle with no gates at all. Re-run: the result is now 100% |0>, meaning CONSTANT.

```
x q[1];
h q[0];
h q[1];
h q[0];
measure q[0] -> c[0];
```

85. Try the other constant oracle f(x) = 1 (an X on q[1] inside the oracle) and the other balanced oracle f(x) = NOT x (CNOT then X on q[1]). Confirm the verdicts are 0 and 1 respectively.

```
x q[1];
h q[0];
h q[1];
cx q[0], q[1];
x q[1];
h q[0];
measure q[0] -> c[0];
```

86. Scale up to Deutsch-Jozsa on 4 qubits: H on q[0..2], X then H on q[3], a balanced oracle of CNOTs from each input to q[3], then H on q[0..2] and measure them.

```
x q[3];
h q[0];
h q[1];
h q[2];
h q[3];
cx q[0], q[3];
cx q[1], q[3];
cx q[2], q[3];
h q[0];
h q[1];
h q[2];
```

87. Run and read the histogram: any non-zero outcome means BALANCED, while an all-zeros outcome (000) would mean CONSTANT.

Test it — how to verify you succeeded

Both constant oracles measure all-zeros with certainty and both balanced oracles measure a non-zero result with certainty. You can state the speedup: one quantum query versus up to 2^(n-1)+1 classical queries.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The OpenQASM for this circuit is also saved in labs/lab-11-the-deutsch-and-deutsch-jozsa-algorithms/.

Lab 12 — The Bernstein-Vazirani Algorithm

Learning outcome: LO3 — perform coding and conduct testing of quantum algorithms.

Goal: Recover a hidden 4-bit string in a single query instead of four. Build the circuit for the hidden string 0101 (decimal 5) until it is second nature.

What you'll build

A 5-qubit Bernstein-Vazirani circuit that reads out the hidden string 0101 deterministically. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

88. Open a 5-qubit circuit: q[0]-q[3] are the input register and q[4] is the ancilla. Add 4 classical bits.
89. Prepare the ancilla in |->: apply X to q[4], then H to q[4].

```
x q[4];
h q[4];
```

90. Apply H to every input qubit q[0], q[1], q[2] and q[3] to create a uniform superposition over all 16 inputs.

```
h q[0];
h q[1];
h q[2];
h q[3];
```

91. Add a barrier, then build the oracle for the hidden string s = 0101. Place a CNOT from each qubit where s has a 1 — that is q[0] and q[2] — onto the ancilla q[4].

```
cx q[0], q[4];
cx q[2], q[4];
```

92. Add a barrier and apply H again to all four input qubits q[0]-q[3].

```
h q[0];
h q[1];
h q[2];
h q[3];
```

93. Measure the four input qubits into the classical register.

```
measure q[0] -> c[0];
measure q[1] -> c[1];
measure q[2] -> c[2];
measure q[3] -> c[3];
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

94. Run 1024 shots. A single bar appears at 0101 with 100% probability — the hidden string, found in ONE query.
95. Change the hidden string to 1101 by adding CNOTs from q[0], q[1] and q[3] to the ancilla, and confirm the readout follows. Note that phase kickback is what writes s onto the input register.

```
x q[4];
h q[4];
h q[0];
h q[1];
h q[2];
h q[3];
cx q[0], q[4];
cx q[1], q[4];
cx q[3], q[4];
h q[0];
h q[1];
h q[2];
h q[3];
```

Test it — how to verify you succeeded

The histogram shows a single 100% bar at the hidden string you encoded. You can explain the oracle construction rule (one CNOT per 1-bit of s) and that a classical algorithm would need n separate queries.

Note: The OpenQASM for this circuit is also saved in labs/lab-12-the-bernstein-vazirani-algorithm/.

Lab 13 — Grover's Search Algorithm

Learning outcome: LO3 — perform coding and conduct testing of quantum algorithms.

Goal: Build the best-known quantum search algorithm on IBM Quantum Composer. Mark a target item with an oracle, amplify it with the diffusion operator, and find it in O(sqrt(N)) steps.

What you'll build

A 2-qubit Grover circuit that finds the marked item with near-certainty, plus a 3-qubit version. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

96. Open a 2-qubit circuit with 2 classical bits. Apply H to q[0] and q[1] to create an equal superposition over |00>, |01>, |10> and |11>, each at 25%.

```
h q[0];
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
h q[1];
```

97. Add a barrier, then build the ORACLE that marks |11> with a phase flip: a single CZ gate between q[0] and q[1].

```
cz q[0], q[1];
```

98. Add a barrier, then build the DIFFUSION operator (inversion about the mean): H on both, X on both, CZ, X on both, H on both.

```
h q[0];
h q[1];
x q[0];
x q[1];
cz q[0], q[1];
x q[0];
x q[1];
h q[0];
h q[1];
```

99. Measure both qubits and run 1024 shots. The marked state |11> appears with essentially 100% probability after just ONE Grover iteration.

```
measure q[0] -> c[0];
measure q[1] -> c[1];
```

100. Change the marked item to |01>: put an X on q[0] before and after the CZ inside the oracle, so the phase flip lands on |01> instead.

```
h q[0];
h q[1];
x q[0];
cz q[0], q[1];
x q[0];
```

101. Re-run and confirm the histogram now peaks at |01|. Repeat for |00> (X on both qubits around the CZ) and |10>.
102. Scale to 3 qubits searching for |111>: H on q[0..2], a CCZ oracle (or H-CCX-H on q[2]), then the 3-qubit diffusion operator, and repeat the iteration TWICE.

```
h q[0];
h q[1];
h q[2];
h q[2];
ccx q[0], q[1], q[2];
h q[2];
```

103. Compare the query counts: Grover needs about sqrt(N) iterations while a classical search needs N/2 on average — a quadratic speedup.

Test it — how to verify you succeeded

For 2 qubits, the marked state is returned with near-100% probability after one iteration, and you can re-target the oracle to any of the four states. You can state why the number of iterations must be about pi/4 * sqrt(N) and what happens if you over-rotate.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

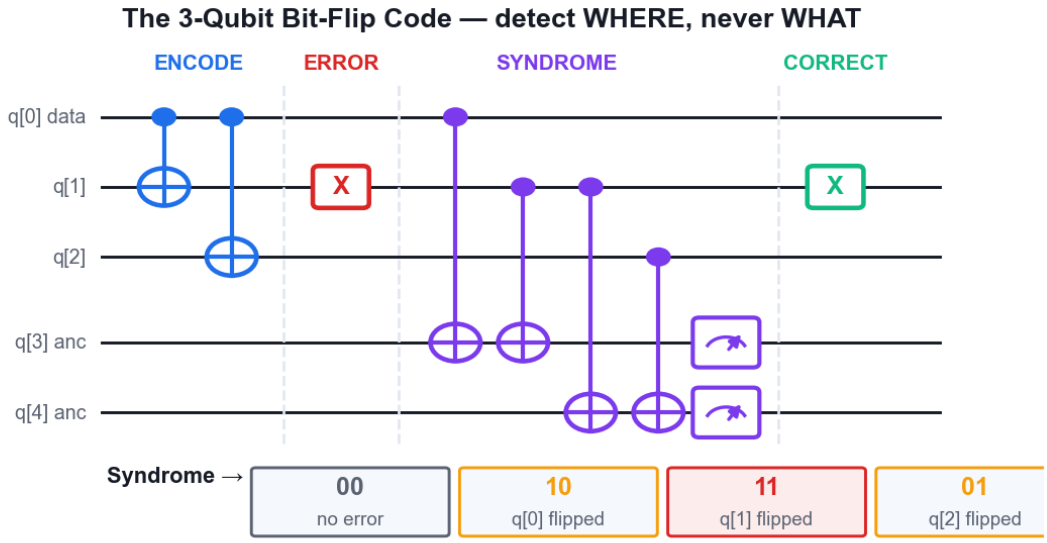
Note: The OpenQASM for this circuit is also saved in labs/lab-13-grovers-search-algorithm/.

Topic 04 — Quantum Computing Applications

Introduction to Quantum Error Correction · Introduction to Quantum Machine Learning

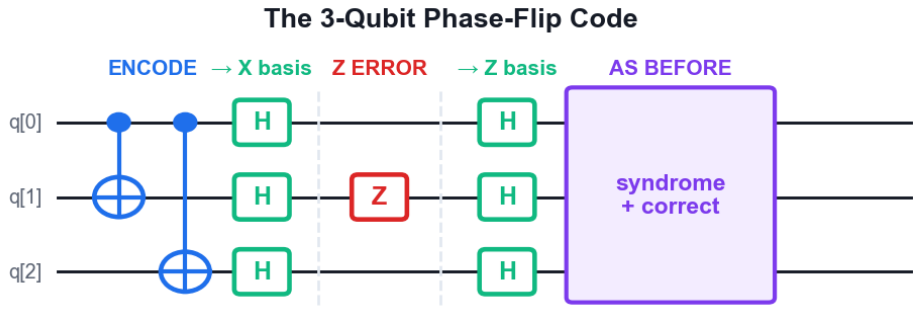
Key concepts

- **Why error correction —** Qubits decohere and gates are imperfect. Without QEC, a long algorithm produces noise, not answers.
- **Bit-flip and phase-flip codes —** Encode one logical qubit into three physical qubits; detect with syndrome measurement, then correct.
- **Shor, Steane and surface codes —** 9-qubit, 7-qubit and 2-D lattice codes that correct arbitrary single-qubit errors.
- **Fault tolerance —** Below the error threshold, adding qubits makes the logical error rate arbitrarily small.
- **Quantum machine learning —** Variational circuits with trainable parameters (VQE, QAOA) run as hybrid quantum-classical loops.
- **Simulator vs real hardware —** A simulator has perfect qubits; real hardware adds decoherence and gate/readout error — results differ.
- **The NISQ era —** Noisy Intermediate-Scale Quantum: 50-1,000 noisy qubits, no fault tolerance yet.
- **Use cases —** Chemistry and drug discovery, logistics and finance optimisation, materials, cryptography and QML.



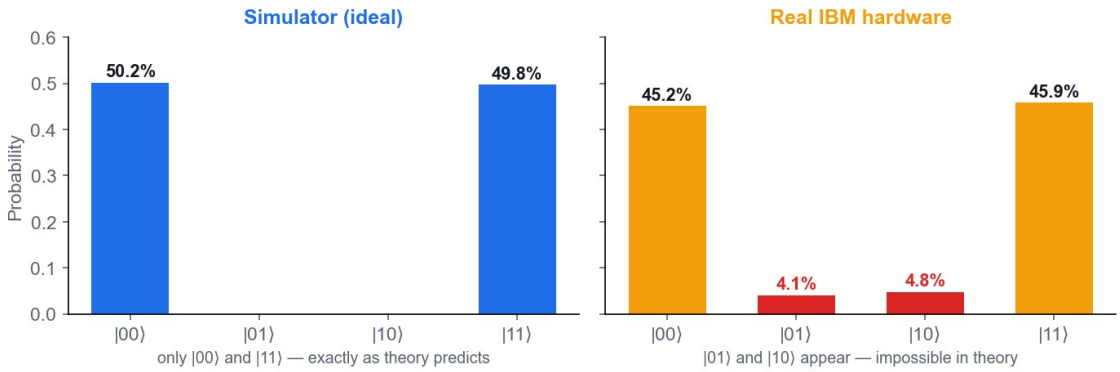
The 3-qubit bit-flip code: encode, error, syndrome, correct.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



The trick: $HZH = X$ — conjugating by Hadamards turns a PHASE flip into a BIT flip

The phase-flip code: H Z H = X turns a phase flip into a bit flip.



The same Bell circuit on the simulator and on real hardware.

Lab 14 — Quantum Error Correction — The Bit-Flip Code

Learning outcome: LO4 — diagnose outcomes and implement changes.

Goal: Protect one logical qubit against a bit-flip error by encoding it into three physical qubits. Deliberately inject an X error on IBM Quantum Composer, detect it with a syndrome measurement, and correct it.

What you'll build

A 3-qubit bit-flip code that detects and corrects an injected X error on any one of the three qubits. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

104. Open a 5-qubit circuit: q[0]-q[2] hold the encoded logical qubit and q[3]-q[4] are syndrome ancillas.
105. ENCODE: place the logical state on q[0] (use H for a superposition, or leave |0>), then CNOT from q[0] to q[1] and from q[0] to q[2]. The logical |0> is now |000> and logical |1> is |111>.

```
h q[0];
cx q[0], q[1];
cx q[0], q[2];
```

106. Add a barrier, then INJECT an error: place a single X gate on q[1] to simulate a bit flip on the middle qubit.

```
x q[1];
```

107. Add a barrier, then measure the first syndrome: CNOT from q[0] to q[3] and CNOT from q[1] to q[3]. This ancilla now holds the parity of qubits 0 and 1.

```
cx q[0], q[3];
cx q[1], q[3];
```

108. Measure the second syndrome: CNOT from q[1] to q[4] and CNOT from q[2] to q[4] — the parity of qubits 1 and 2.

```
cx q[1], q[4];
cx q[2], q[4];
```

109. Measure both ancillas and run. The syndrome reads 11, which uniquely identifies q[1] as the flipped qubit. Syndrome 10 means q[0], 01 means q[2], and 00 means no error.

```
measure q[3] -> c[0];
measure q[4] -> c[1];
```

110. CORRECT the error by applying an X gate to q[1], then verify the logical state is restored by decoding (CNOT q[0]->q[2], CNOT q[0]->q[1], then H on q[0]) and measuring q[0].

```
x q[1];
cx q[0], q[2];
cx q[0], q[1];
h q[0];
measure q[0] -> c[2];
```

111. Move the injected X to q[0] and then to q[2], and confirm the syndrome changes to 10 and 01 respectively — the code locates the error without ever measuring the logical state itself.

Test it — how to verify you succeeded

Each single-qubit X error produces its own distinct syndrome, and after correction the decoded logical qubit returns its original value. You can explain why the syndrome measurement does not collapse the logical superposition.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The OpenQASM for this circuit is also saved in labs/lab-14-quantum-error-correction-—-the-bit-flip-/.

Lab 15 — Quantum Error Correction — The Phase-Flip Code

Learning outcome: LO4 — diagnose outcomes and implement changes.

Goal: Phase flips are invisible to the bit-flip code. Learn the standard trick — surround the code with Hadamards to turn Z errors into X errors — and correct a phase flip on IBM Quantum Composer.

What you'll build

A 3-qubit phase-flip code that detects and corrects an injected Z error. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

112. First show the problem: build the bit-flip code from Lab 14 but inject a Z error instead of an X. The syndrome reads 00 — the error is completely undetected.

```
cx q[0], q[1];
cx q[0], q[2];
z q[1];
```

113. Now build the phase-flip code. ENCODE as before with two CNOTs, then add an H to EACH of q[0], q[1] and q[2]. This moves the code into the X basis, where |+++> and |---> are the logical states.

```
cx q[0], q[1];
cx q[0], q[2];
h q[0];
h q[1];
h q[2];
```

114. Add a barrier and INJECT a Z error on q[1] — a phase flip on the middle qubit.

```
z q[1];
```

115. Add a barrier, then apply H to all three code qubits again. This converts the Z error into an X error, which the bit-flip machinery can see.

```
h q[0];
h q[1];
h q[2];
```

116. Run the same syndrome extraction as Lab 14: CNOTs from q[0] and q[1] onto ancilla q[3], and from q[1] and q[2] onto ancilla q[4].

```
cx q[0], q[3];
cx q[1], q[3];
cx q[1], q[4];
cx q[2], q[4];
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

117. Measure the ancillas and run. The syndrome now reads 11, correctly identifying q[1] — the phase flip has become visible.

```
measure q[3] -> c[0];
measure q[4] -> c[1];
```

118. Correct by applying X to q[1] (in this rotated basis), then decode and verify the logical qubit is intact.

```
x q[1];
```

119. Note how the Shor 9-qubit code combines both: three phase-flip blocks, each of three bit-flip qubits, correcting an ARBITRARY single-qubit error.

Test it — how to verify you succeeded

A Z error is invisible to the bit-flip code (syndrome 00) but produces syndrome 11 in the phase-flip code. You can explain the H-conjugation trick: H Z H = X.

Note: The OpenQASM for this circuit is also saved in labs/lab-15-quantum-error-correction-—-the-phase-fli/.

Lab 16 — Simulator vs Real Quantum Hardware

Learning outcome: LO4 — diagnose unintended outcomes and propose changes.

Goal: Run the same circuit on the ideal simulator and on real IBM quantum hardware, compare the histograms, and diagnose the difference.

What you'll build

A side-by-side comparison of the Bell-state histogram on the simulator and on real hardware, with the error sources identified. (Platform: IBM Quantum Composer, IBM Quantum Platform — https://quantum.cloud.ibm.com/composer)

Step-by-step

120. Rebuild the Bell-state circuit from Lab 3: H on q[0], CNOT(q[0], q[1]), measure both.

```
h q[0];
cx q[0], q[1];
measure q[0] -> c[0];
measure q[1] -> c[1];
```

121. Run it on the SIMULATOR with 1024 shots and record the exact counts. You should see only |00> and |11>, each near 512.
122. Switch the backend to a real IBM quantum device from the backend selector, and submit the same circuit with 1024 shots.
123. Wait for the job to complete in the queue, then open the results and record the counts for all four outcomes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

124. Compare: the hardware histogram still peaks at |00> and |11>, but now |01> and |10> have non-zero counts — these are errors that are impossible in theory.
125. Identify the error sources: decoherence (T1 energy relaxation and T2 dephasing), imperfect two-qubit gate fidelity, readout/measurement error, and crosstalk between neighbouring qubits.
126. Open the backend's calibration data on the IBM Quantum Platform and note the reported CNOT error rate, readout error and T1/T2 times for the qubits you used.
127. Propose mitigations: use fewer gates and shallower circuits, pick qubits with the best calibration, increase shot counts, apply readout-error mitigation, and ultimately use quantum error correction.

Test it — how to verify you succeeded

You can quantify the discrepancy (the percentage of shots landing in the forbidden |01> and |10> states on hardware) and name at least three physical causes plus three practical mitigations.

Note: The OpenQASM for this circuit is also saved in labs/lab-16-simulator-vs-real-quantum-hardware/.

Lab 17 — Introduction to Quantum Machine Learning

Learning outcome: LO4 — propose and implement changes for quantum computing applications.

Goal: Build a parameterised variational circuit — the building block of every quantum machine learning model. Vary the parameters on IBM Quantum Composer, observe the effect on the output distribution, and see how a hybrid quantum-classical training loop works.

What you'll build

A 2-qubit variational ansatz whose rotation angles you tune to steer the output distribution towards a target. (Platform: IBM Quantum Composer — https://quantum.cloud.ibm.com/composer)

Step-by-step

128. Understand the pattern: a variational circuit has a data-encoding layer, a trainable rotation layer, an entangling layer, and a measurement that feeds a classical optimiser.
129. Open a 2-qubit circuit. Build the ENCODING layer: an RY gate on each qubit with the angle set from your input data (use pi/4 on q[0] and pi/3 on q[1]).

```
ry(pi/4) q[0];
ry(pi/3) q[1];
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

130. Add the ENTANGLING layer: a CNOT with control q[0] and target q[1]. This lets the model capture correlations between features.

```
cx q[0], q[1];
```

131. Add the TRAINABLE layer: an RY gate on each qubit with variational parameters — start with theta = pi/2 on both.

```
ry(pi/2) q[0];
ry(pi/2) q[1];
```

132. Measure both qubits and run 1024 shots. Record the four output probabilities — this is the model's current prediction.

```
measure q[0] -> c[0];
measure q[1] -> c[1];
```

133. Now train by hand: change the trainable angles to pi/4, re-run, and see whether the probability of your target outcome went up or down. This is one step of the optimiser's job.

```
ry(pi/4) q[0];
ry(pi/4) q[1];
```

134. Repeat for at least four angle settings and tabulate the target-outcome probability against the angle. Pick the best setting — you have just performed a manual parameter sweep.
135. Relate this to real QML: VQE minimises a molecule's ground-state energy and QAOA maximises a combinatorial objective, both by running exactly this loop with a classical optimiser choosing the next angles.

Test it — how to verify you succeeded

You can produce a table of target-outcome probability versus rotation angle and identify the best angle you tested. You can name the four parts of a variational circuit and explain why the loop is called hybrid quantum-classical.

Note: The OpenQASM for this circuit is also saved in labs/lab-17-introduction-to-quantum-machine-learning/.

Troubleshooting — Common Problems

- **My histogram shows the bits reversed —** IBM orders the label with the highest classical bit on the LEFT (c[1]c[0]). Read the string right-to-left if you are matching it against q[0], q[1], …
- **My circuit will not run —** Check that every measured qubit has a Measure operation and that the classical register is large enough. A control and its target must be on DIFFERENT wires.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **I get 50/50 when I expected 100% —** Usually a missing final Hadamard layer, or an oracle placed on the wrong qubit. Compare your OpenQASM line-by-line with the listing in the lab.
- **The statevector panel is empty —** The statevector is only shown for circuits WITHOUT a measurement. Delete the Measure operations temporarily to inspect the amplitudes, then put them back to run.
- **Grover returns the wrong state —** Check the X gates around your CZ — they select WHICH basis state is marked. Also check your iteration count: for N = 4 exactly one iteration is correct, and more will make it worse.
- **My hardware job is stuck in the queue —** Real devices are shared. Choose a backend with a shorter queue, or use the simulator for everything except Lab 16.
- **Results differ every time on hardware —** That is expected. Decoherence, gate infidelity and readout error make each run statistically different — see Lab 16.

Glossary

- **Qubit —** The quantum unit of information. Unlike a bit it can be in a superposition of |0> and |1>.
- **Superposition —** A state that is a weighted combination of basis states, existing as both until measured.
- **Amplitude —** The complex number multiplying a basis state. Its squared magnitude is the measurement probability.
- **Bloch sphere —** The geometric picture of a single-qubit pure state as a point on the surface of a sphere.
- **Entanglement —** A correlation between qubits so strong that their states cannot be described independently.
- **Bell state —** One of the four maximally entangled two-qubit states, e.g. (|00> + |11>)/sqrt(2).
- **GHZ state —** The n-qubit generalisation of a Bell state, (|0...0> + |1...1>)/sqrt(2).
- **Unitary —** A reversible, norm-preserving operation. Every quantum gate is unitary.
- **Hadamard (H) —** The gate that creates superposition: H|0> = |+>. It is its own inverse.
- **Pauli gates (X, Y, Z) —** The three fundamental single-qubit gates: bit flip, bit-and-phase flip, and phase flip.
- **CNOT —** The controlled-NOT gate — flips the target when the control is |1>. The standard entangling gate.
- **Toffoli (CCNOT) —** A 3-qubit gate that flips the target when both controls are |1>; universal for reversible classical logic.
- **Phase kickback —** The effect where a controlled gate's phase appears on the CONTROL qubit; the engine of the query algorithms.
- **Oracle —** A black-box unitary encoding an unknown function, queried by the algorithm.
- **No-cloning theorem —** It is impossible to copy an arbitrary unknown quantum state.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Quantum teleportation —** Transferring an unknown qubit state using a shared Bell pair and two classical bits.
- **Superdense coding —** Sending two classical bits by transmitting one qubit of a shared Bell pair.
- **QKD / BB84 —** Quantum Key Distribution; BB84 (1984) is the first and best-known protocol.
- **Grover's algorithm —** Unstructured search in about sqrt(N) queries — a quadratic speedup.
- **Deutsch-Jozsa —** Decides whether a function is constant or balanced in a single query.
- **Bernstein-Vazirani —** Recovers an n-bit hidden string in one query instead of n.
- **QFT —** Quantum Fourier Transform — the basis of phase estimation and Shor's algorithm.
- **Decoherence —** The loss of quantum information to the environment; characterised by the T1 and T2 times.
- **Syndrome measurement —** Measuring parity via ancilla qubits to locate an error WITHOUT collapsing the logical state.
- **QEC —** Quantum Error Correction — encoding one logical qubit across several physical qubits.
- **Fault tolerance —** Designing computation so errors do not spread and can be corrected indefinitely.
- **NISQ —** Noisy Intermediate-Scale Quantum — today's era of 50-1,000 noisy, uncorrected qubits.
- **Ansatz —** The parameterised circuit template trained in a variational quantum algorithm.
- **VQE / QAOA —** Variational Quantum Eigensolver (chemistry) and Quantum Approximate Optimisation Algorithm.
- **Shot —** One execution-and-measurement of a circuit. Results are aggregated over many shots (typically 1024).
- **OpenQASM —** The open quantum assembly language used by IBM Quantum Composer to represent a circuit as text.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg